```
coffee = require 'coffee-script'

expression = 'I = (x) -> x'

coffee.tokens expression
#   [[ 'IDENTIFIER', 'I'],
#   [ '=', '='],
#   [ 'PARAM_START', '('],
#   [ 'IDENTIFIER', 'x'],
#   [ 'PARAM_END', ')'],
#   [ '->', '->'],
#   [ 'INDENT', 2],
#   [ 'IDENTIFIER', 'x'],
#   [ 'OUTDENT', 2],
#   [ 'TERMINATOR', '\n'] ]
```

These tokens have been modified to show only the token types and values.

The lexer has generated tokens that represent the string of CoffeeScript code it was invoked with. Here's a visual representation of how the tokens are broken down:



When tokenizing, it's common for a compiler to ignore some things like comments and specific characters by not generating any tokens for them. Here, whitespace in the middle of the line has been ignored. In addition to skipping characters, the CoffeeScript compiler also modifies the token stream by adding tokens. Notice that the `INDENT` and `OUTDENT` tokens don't correspond to any indentation in the original source string. The *rewriter* has added these tokens.

8.3.2. Rewriting

The myriad of syntax options in CoffeeScript means that tokenizing and building an abstract syntax tree for everything is a complicated task. Because some of the syntax is mostly convenience, CoffeeScript also has a rewriter that will modify the token stream. The rewriting done by the compiler is described here.